

Guion-Runbook para grabar el video del Sumativo

Plan **cronológico** pensado para ejecutarse y grabarse en **una sola sesión**, partiendo de un **Laboratorio AWS Academy recién iniciado y sin nada descargado**. Sigue las fases en orden: lo de la **Fase 0 hazlo ANTES de grabar** (no aporta al video y consume tiempo del lab); desde la **Fase 1 enciende la cámara**.

🚨 **Importante (AWS Academy 2026):** las credenciales del Learner Lab son **temporales** y caducan al detener el lab o tras unas horas. Crea S3/EFS/EC2, configura y graba **sin cerrar el lab**. Si caducan a mitad de camino, vuelve a copiar las credenciales (Fase 1) y actualiza los GitHub Secrets + reinicia el contenedor.

FASE 0 — Preparativos locales (ANTES de grabar)

Objetivo: Llegar a la grabación con el código en GitHub, las cuentas listas y todo probado, para que el video sea fluido.

0.1 Cuentas necesarias

- ☐ Cuenta de **GitHub** (repo donde subirás **transportes-guias-api**).
- ☐ Cuenta de **Docker Hub** + un **Access Token** (Account Settings → Security → New Access Token).
- ☐ Acceso al **Learner Lab** de AWS Academy.

0.2 Software local

- ☐ **Git, JDK 21, Maven, Docker Desktop, Postman**.
- ☐ Grabador de pantalla (**OBS Studio** o similar) con micrófono probado.

0.3 Probar el proyecto localmente (en **transportes-guias-api**)

```
mvn clean package
```

Debe compilar y pasar el test. (Opcional: corre el jar y prueba 1 request en Postman.)

0.4 Subir el proyecto a GitHub (si quedó un **.git/target** previo, bórralos primero)

```
Remove-Item -Recurse -Force .git, target -ErrorAction SilentlyContinue
git init
git add .
git commit -m "V2 Grupo13: solucion Cloud Native"
git branch -M main
git remote add origin https://github.com/TU_USUARIO/transportes-guias-api.git
git push -u origin main
```

No configures aún los Secrets si no quieres que el push dispare el deploy en vacío. El pipeline compilará y publicará la imagen igual; el job de deploy fallará hasta tener los Secrets de EC2. Alternativa: deja los Secrets para la Fase 6 y **re-ejecuta** el workflow allí.

0.5 Guion en pantalla

- ☐ Ten abiertos en pestañas: Learner Lab, consola AWS (S3, EFS, EC2), repo de GitHub (pestaña Actions), Docker Hub, Postman, una terminal y el editor con el código.

FASE 1 — (CÁMARA ON) Iniciar el Lab y obtener credenciales — *contexto*

1. Inicia el Learner Lab → espera el círculo **verde** junto a "AWS" → clic en **AWS** para abrir la consola.
2. Clic en **AWS Details** → **AWS CLI**: copia **aws_access_key_id**, **aws_secret_access_key** y **aws_session_token**. Región por defecto: **us-east-1**.
3. En **AWS Details** descarga la llave **labsuser.pem** (keypair **vockey**) — la usarás para SSH/EC2.

🗣 *Qué decir:* "Estas son credenciales **temporales** del Learner Lab; nunca van en el código, las cargaré como variables de entorno y GitHub Secrets."

FASE 2 — Crear el bucket S3 (privado) — *Criterio 2 (10 pts)*

1. Consola → **S3** → **Create bucket**.
2. Nombre único, p. ej. **transportes-guias-grupo13**. Región **us-east-1**.
3. **Block all public access** = **ACTIVADO** (bucket privado). Crear.

🗣 *Qué decir:* "El bucket es el almacenamiento **final** y se mantiene **privado**."

FASE 3 — Crear Amazon EFS — *Criterio 1 (15 pts)*

1. Consola → **EFS** → **Create file system** → **Customize**.
2. VPC: la **default** (la misma donde lanzarás la EC2). Crear.
3. Anota el **File system ID** (**fs-xxxxxxx**).
4. **Network**: en el security group del EFS asegúrate (o créalo en Fase 4) de permitir **NFS (2049)** desde el security group de la EC2. (*Tip: puedes usar el SG default de la VPC para EFS y EC2 y permitir 2049 dentro del mismo SG.*)

🗣 *Qué decir:* "EFS será el almacenamiento **temporal**: la guía se genera primero aquí y luego sube a S3."

FASE 4 — Lanzar la instancia EC2 — *base del Criterio 6*

1. Consola → **EC2** → **Launch instance**.
2. Nombre: **ec2-transportes-guias**. AMI: **Amazon Linux 2023**. Tipo: **t2.micro / t3.micro**.
3. **Key pair**: selecciona **vockey** (la de **labsuser.pem**).
4. **Network settings** → **Security group**, permitir entrada:
 - **SSH (22)** desde *My IP* (o 0.0.0.0/0 para la demo).

- **TCP 8080** desde 0.0.0.0/0 (API).
 - Para EFS: que este SG (o el de EFS) permita **NFS 2049** entre EC2 y EFS.
5. **Advanced** → **IAM instance profile**: selecciona **LabInstanceProfile** (rol **LabRole**).
6. Launch. Anota la **IP pública** (o DNS público).

 *Qué decir:* "La EC2 ejecutará el contenedor; abro 22 para el deploy por SSH y 8080 para la API."

FASE 5 — Configurar la EC2: Docker + montar EFS en /mnt/efs — *Criterio 1*

Conéctate por SSH (en Windows, desde PowerShell en la carpeta donde está **labsuser.pem**):

```
icaccls labsuser.pem /inheritance:r ; icaccls labsuser.pem /grant:r
"$($env:USERNAME):(R)"
ssh -i labsuser.pem ec2-user@TU_IP_EC2
```

Ya dentro de la EC2:

```
# Docker
sudo yum update -y
sudo yum install -y docker amazon-efs-utils
sudo systemctl enable --now docker
sudo usermod -aG docker ec2-user
# (re-conéctate por SSH para aplicar el grupo docker)

# Montar EFS
sudo mkdir -p /mnt/efs
sudo mount -t efs -o tls fs-XXXXXXXX:/ /mnt/efs

# Verificar (ESTO va al video)
df -h | grep efs
```

 *Qué mostrar:* **df -h | grep efs** con **/mnt/efs** montado → evidencia del **Criterio 1 (15 pts)**.

FASE 6 — Configurar GitHub Secrets — *habilita el Criterio 6*

En GitHub: repo → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**. Crea:

Secret	Valor
DOCKERHUB_USERNAME	tu usuario de Docker Hub
DOCKERHUB_TOKEN	el access token de Docker Hub
EC2_HOST	IP pública de la EC2

Secret	Valor
EC2_USER	ec2-user
EC2_SSH_KEY	contenido completo de labsuser.pem
AWS_REGION	us-east-1
AWS_ACCESS_KEY_ID	de la Fase 1
AWS_SECRET_ACCESS_KEY	de la Fase 1
AWS_SESSION_TOKEN	de la Fase 1
AWS_S3_BUCKET	transportes-guias-grupo13

🔊 *Qué decir:* "Las credenciales viven aquí como Secrets, **no en el repositorio.**"

FASE 7 — Disparar el pipeline (CI/CD) — Criterio 6 (20 pts)

Opción A (recomendada en cámara): hacer un cambio mínimo y push.

```
git commit --allow-empty -m "Deploy demo Grupo13"
git push origin main
```

Opción B: GitHub → **Actions** → workflow → **Run workflow**.

Muestra en el video, en este orden:

1. **GitHub** → **Actions**: jobs **build-and-push** y **deploy-ec2** en **verde**.
2. **Docker Hub**: imagen **TU_USUARIO/transportes-guias-api** con tags **latest** y **<sha>**.
3. **EC2 (SSH)**:

```
docker ps
docker inspect transportes-guias-api --format '{{.Mounts}}' # muestra
-v /mnt/efs:/app/efs
curl http://localhost:8080/actuator/health
```

🔊 *Qué decir:* "El push construye la imagen, la publica en Docker Hub y por SSH la despliega en EC2 con el volumen **/mnt/efs:/app/efs** montado." → **Criterio 6 (20 pts)**.

FASE 8 — Demostración funcional con Postman + evidencias

Importa **docs/postman/Transportes-Guias.postman_collection.json** y pon **baseUrl** = **http://TU_IP_EC2:8080**. Ejecuta en este orden (cada paso es un criterio de la pauta):

1. **Crear guía** — **POST** **/api/guias** → **201**, estado **GENERADA**, **efsPath** poblado.

- En la EC2: `sudo find /mnt/efs/guias -type f` → el PDF **ya está en EFS** → **Criterio 1 (15)**.
 - 2. **Subir a S3** — `POST /api/guias/{id}/subir-s3` → 200, estado `SUBIDA_S3`.
 - En consola S3: objeto en `guias/<fecha>/<transportista>/...pdf` → **Criterio 2 (10)**.
 - 3. **Descargar** — `GET /api/guias/{id}/descargar`:
 - `X-Usuario-Rol: ADMIN` → 200 (PDF).
 - `X-Usuario-Rol: TRANSPORTISTA` + nombre correcto → 200.
 - `X-Usuario-Rol: TRANSPORTISTA` + nombre ajeno → 403 → **Criterio 4 (10)**.
 - 4. **Actualizar** — `PUT /api/guias/{id}` → 200, estado `ACTUALIZADA`.
 - En S3: el objeto cambia su fecha de modificación (reemplazado) → **Criterio 3 (15)**.
 - 5. **Historial** — `GET /api/guias?transportista=...&fecha=2026-...` → lista filtrada → **Criterio 5 (10)**.
 - 6. **Detalle** — `GET /api/guias/{id}` → datos de la guía.
 - 7. **Eliminar** — `DELETE /api/guias/{id}` → 200, estado `ELIMINADA`; el objeto desaparece de S3.
-

FASE 9 — Cierre (30–60 s)

- Recorre rápido el código por capas y abre `application.yml` / `S3Config` para enfatizar **cero credenciales hardcodeadas**.
 - Repite el mapeo: EFS 15 · S3 subida 10 · S3 modificación 15 · descarga 10 · historial 10 · pipeline 20 · video 20.
 - Menciona la consideración de **credenciales temporales** de AWS Academy (`AWS_SESSION_TOKEN`).
-

Checklist exprés antes de pulsar REC

- ☐ Proyecto en GitHub y `mvn clean package` OK (Fase 0).
- ☐ Cuentas Docker Hub + token, GitHub listos.
- ☐ Learner Lab iniciado (verde) y credenciales a mano.
- ☐ Pestañas abiertas: Lab, S3, EFS, EC2, GitHub Actions, Docker Hub, Postman, terminal, editor.
- ☐ Postman con `baseUr1` apuntando a la EC2.
- ☐ Micrófono y grabador probados.

Plan B si algo falla en cámara

- **Credenciales expiran:** vuelve a la Fase 1, actualiza Secrets y re-ejecuta el workflow; en EC2 `docker rm -f transportes-guias-api` y vuelve a correr el `docker run` (ver `EC2-EFS-SETUP.md`).
- **El job `deploy-ec2` falla por SSH:** verifica `EC2_HOST` (IP actual), `EC2_USER=ec2-user`, y que `EC2_SSH_KEY` tenga el `labsuser.pem` completo (incluidas las líneas BEGIN/END).
- **EFS no monta:** confirma que el SG permite NFS **2049** entre EC2 y EFS y que instalaste `amazon-efs-utils`. Como respaldo demuestra EFS con el `docker run -v /mnt/efs:/app/efs` manual.
- **Pipeline en rojo pero quieres mostrar el deploy:** usa el despliegue **manual** de `EC2-EFS-SETUP.md` (Fase 5) y deja el pipeline como evidencia de build/push.